
Introduction to Parallel Programming

Lecture: CUDA

谭光明

高性能计算机研究中心

Contents

1. Overview
2. CUDA Basics
3. Kernels
4. Threads and thread blocks
5. Communicating data between host and device
6. Data sharing among threads in the device
7. Choosing a block size

Contents

1. **Overview**
2. CUDA Basics
3. Kernels
4. Threads and thread blocks
5. Communicating data between host and device
6. Data sharing among threads in the device
7. Choosing a block size

Goal

- Learn *CUDA*, the basic API for programming NVIDIA GPUs
- Learn where it is similar to OpenMP and where it is different

CUDA reference

- Official documentation

- <https://docs.nvidia.com/cuda/index.html>

- book Professional CUDA C Programming

- <https://www.amazon.com/Professional-CUDA-Programming-John-Cheng/dp/1118739329>

Compiling/running CUDA programs with NVCC

- compile with nvcc command

```
Linux> nvcc program.cu
```

- the conventional extension of CUDA programs is .cu
- nvcc can handle ordinary C/C++ programs too (.cc, .cpp → C+)
- you can have a file with any extension and insist it is a CUDA program (convenient when you maintain a single file that compiles both on CPU and GPU)

```
Linux> nvcc -x cu program.cc
```

Contents

1. Overview
2. **CUDA Basics**
3. Kernels
4. Threads and thread blocks
5. Communicating data between host and device
6. Data sharing among threads in the device
7. Choosing a block size

GPU is a device separate from CPU

As such,

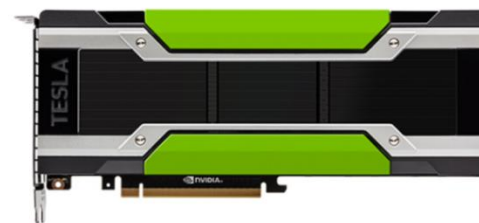
- code (functions) that runs on GPU must be so designated
- data must be copied between CPU and GPU
- a GPU is often called a **"device"**
- and a CPU a **"host"**

host (CPU)

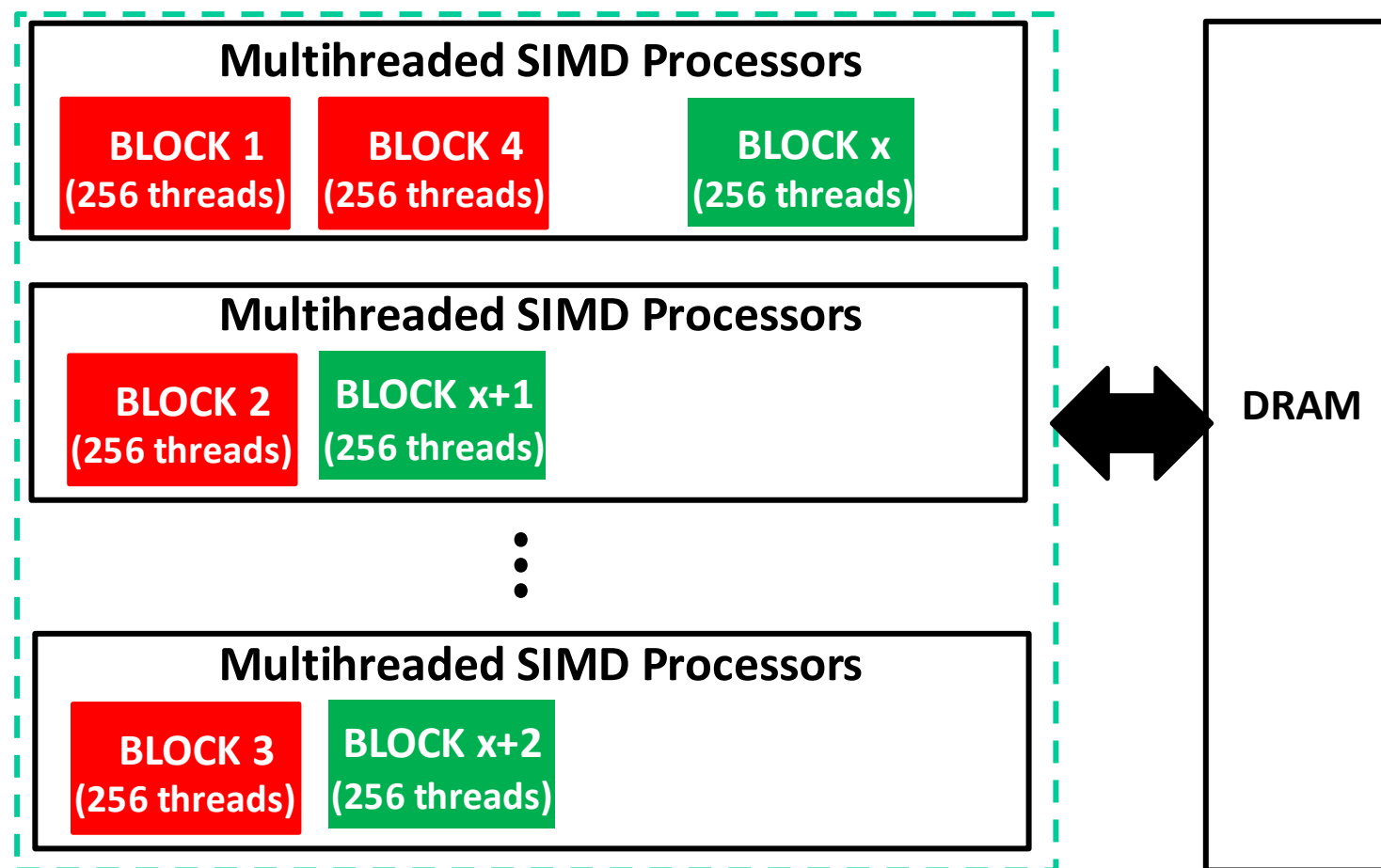


Intel® Core™ i7 processor

device (GPU)



Programmer's View of Execution

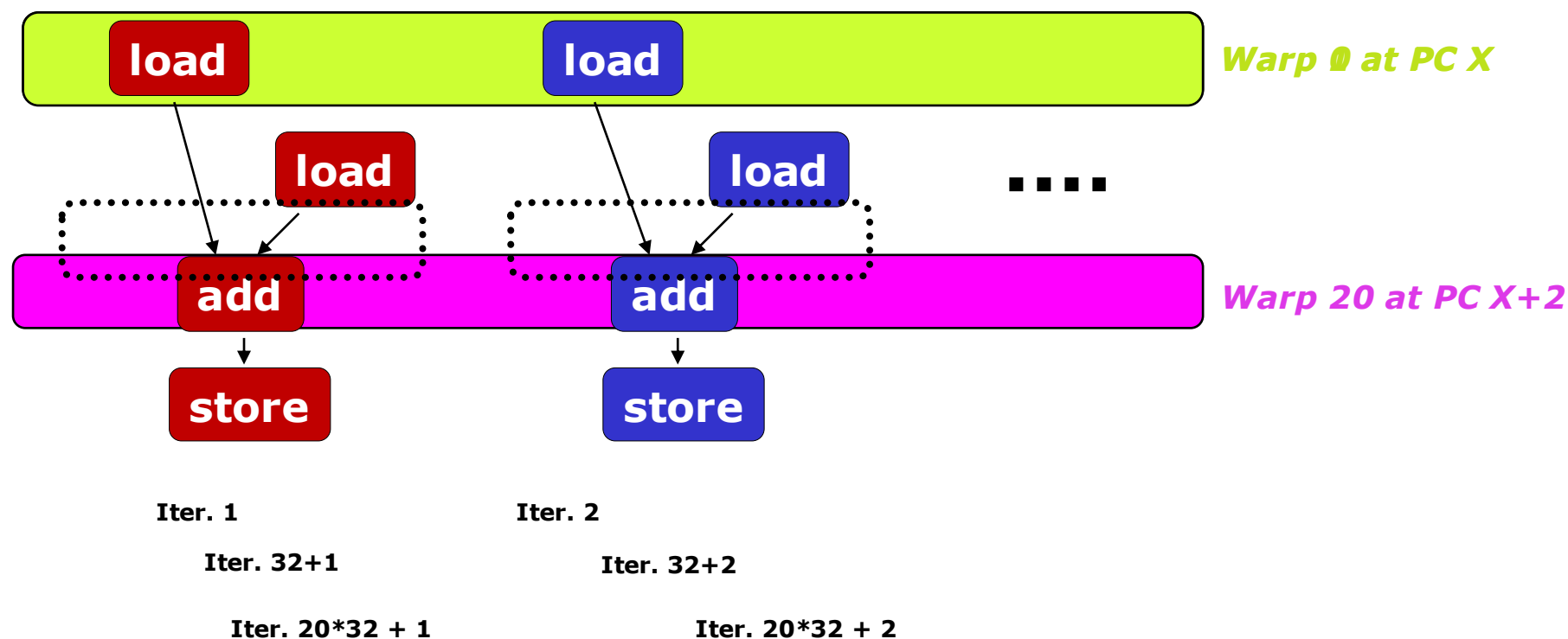


- 创建足够的线程块以适应输入向量
- 多个线程组成线程块**block**，多个线程块构成的、在**GPU**上运行的代码称为**Grid**
- **Also MIMD !**

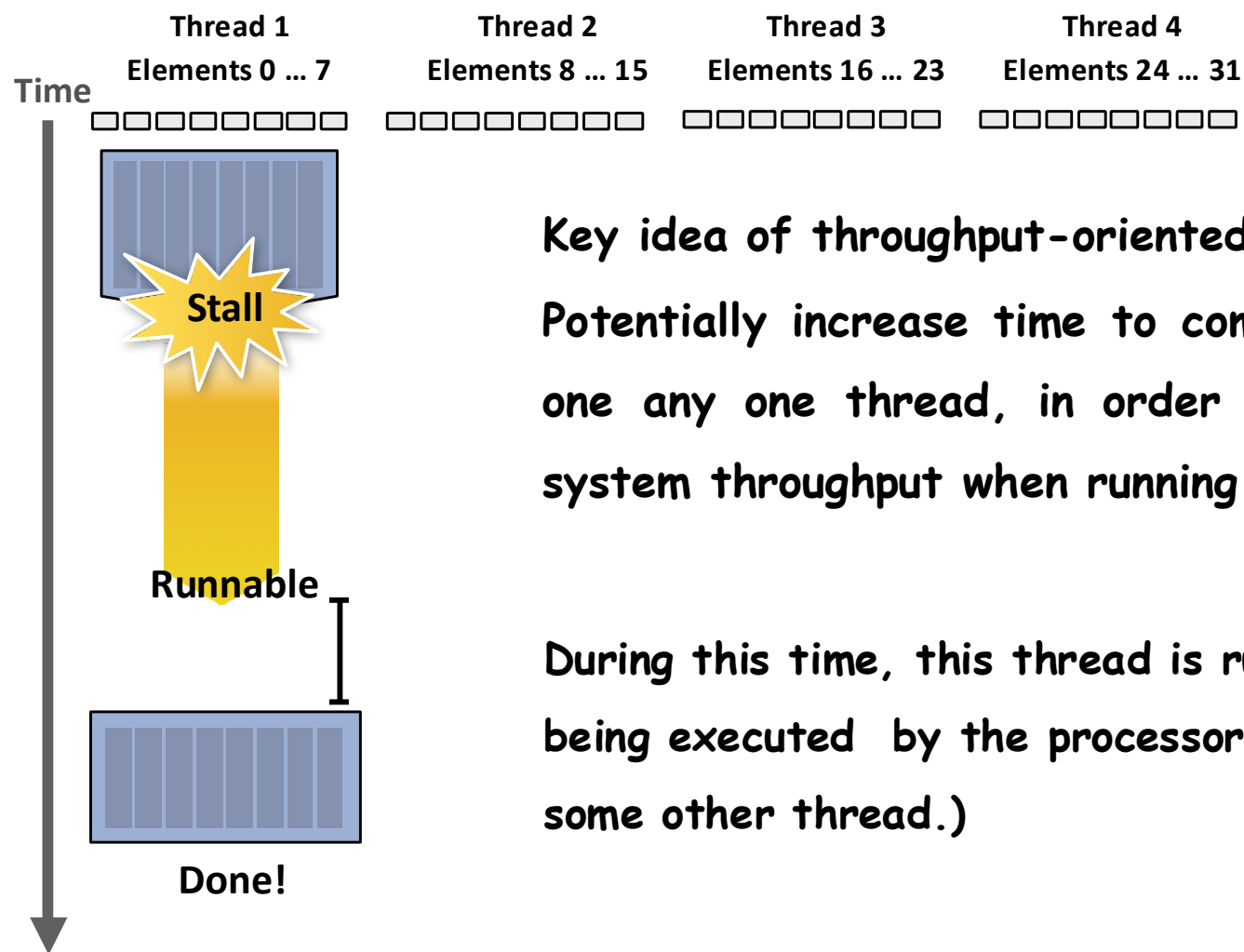
Multithreading of Warps

- 设一个warp由 32 threads构成
- 如果有32K次循环 $\rightarrow$ 1K 个warps
- 这些Warps可以在同一条流水线上交替执行
 - Fine grained multithreading of warps

```
for (i=0; i < N; i++)
    C[i] = A[i] + B[i];
```



Throughput computing trade-off



Key idea of throughput-oriented systems:

Potentially increase time to complete work by any one any one thread, in order to increase overall system throughput when running multiple threads.

During this time, this thread is runnable, but it is not being executed by the processor. (The core is running some other thread.)

Hardware-supported multi-threading

■ Core manages execution contexts for multiple threads

- Runs instructions from runnable threads (processor makes decision about which thread to run each clock, not the operating system)
- Core still has the same number of ALU resources: multi-threading only helps use them more efficiently in the face of high-latency operations like memory access

■ Interleaved multi-threading (a.k.a. temporal multi-threading)

- What I described on the previous slides: each clock, the core chooses a thread, and runs an instruction from the thread on the ALUs

■ Simultaneous multi-threading (SMT)

- Each clock, core chooses instructions from multiple threads to run on ALUs
- Extension of superscalar CPU design
- Example: Intel Hyper-threading (2 threads per core)

Contents

1. Overview
2. CUDA Basics
3. **Kernels**
4. Threads and thread blocks
5. Communicating data between host and device
6. Data sharing among threads in the device
7. Choosing a block size

Two things you need to learn first: writing and launching kernels

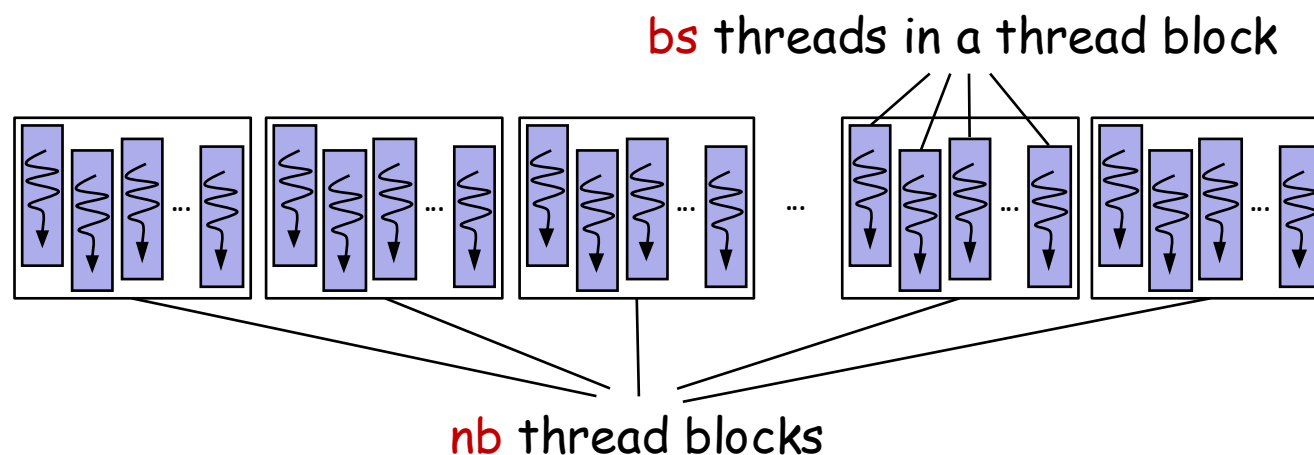
- a “GPU kernel” (or simply a “kernel”) is a function that runs on GPU

```
__global__ void f(...){...}
```

- syntactically, a kernel is an ordinary C++ function that returns nothing (void), except for the **__global__** keyword
- a host launches a kernel specifying the number of threads

```
f<<<nb,ns>>>(...);
```

- will create $(nb \times bs)$ CUDA threads, each executing $f(\dots)$



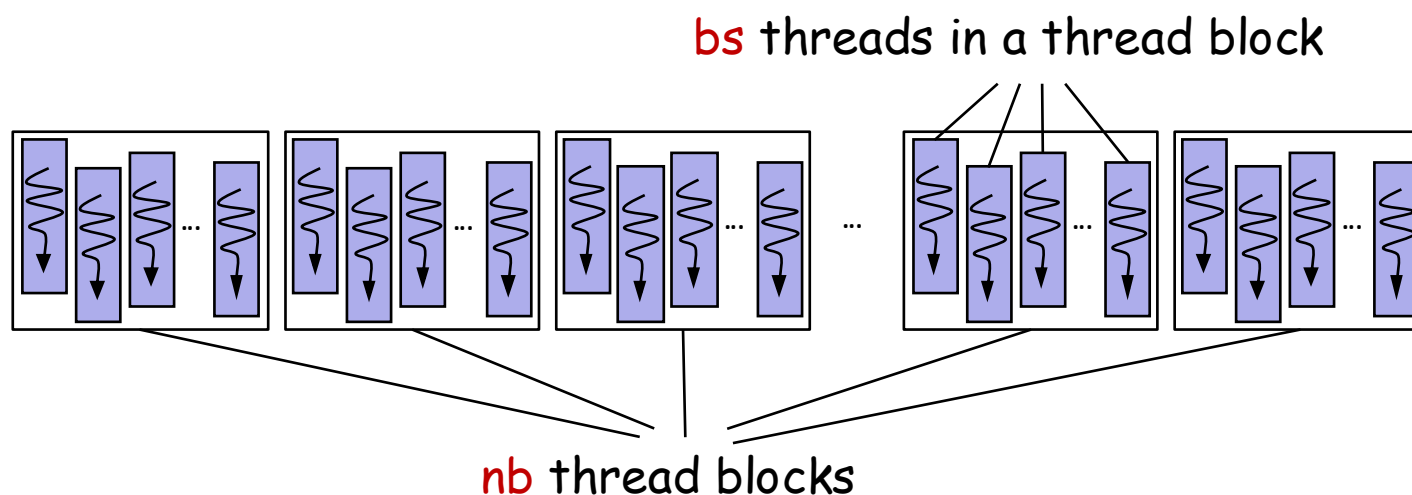
Launching a kernel $\approx$ parallel loop

- launching a kernel, like

```
f<<<nb,ns>>>(...);
```

- $\approx$ executing the following loop in parallel (on GPU, of course)

```
for (i=0;i<nb*ns;i++){  
    f(...); //CUDA thread  
}
```



A simplest example

■ writing a kernel

```
__global__ void cuda_thread_fun (int n){  
    int i = blockDim.x * blockIdx.x + threadIdx.x;  
    int nthreads = gridDim.x * blockDim.x;  
    if (i<n){  
        printf("hello I am CUDA thead %d out of %d\n",i,n);  
    }  
}
```

■ and lunching it

```
int thread_block_sz = 64;  
int n_thread_blocks = (n + thread_block_sz - 1) / thread_block_sz;  
cuda_thread_fun<<<n_thread_blocks,thread_block_sz>>>(n);
```

■ will print hello n times

```
hello I am CUDA thread 0 out of n  
...  
hello I am CUDA thread n - 1 out of n
```

■ note: the order is unpredictable

A CUDA thread is not like an OpenMP thread

- launching 10000 CUDA threads is quite common and efficient

```
f<<<1024,256>>>(...);
```

- launching 10000 threads on CPU is almost always a bad idea
- below is “semantically” similar to the above

```
/* pragma omp parallel */  
f();
```

```
Linux> OMP NUM THREADS=262144 ./a.out
```

- but what happens inside is very different
- CPU way of doing this was:

```
/* pragma omp parallel for */  
for (i = 0; i < 1024 * 256; i++) { f(); }
```

```
Linux> OMP NUM THREADS=a modest number ./a.out
```

- a modest number = typically the actual number of cores

A kernel call and the host overlap but two kernel calls do not

- when you call a kernel, the host continues execution without waiting for it to finish
- two kernel calls are serialized on the GPU side, by default
- `cudaDeviceSynchronize()` is an API to wait for the kernel to finish

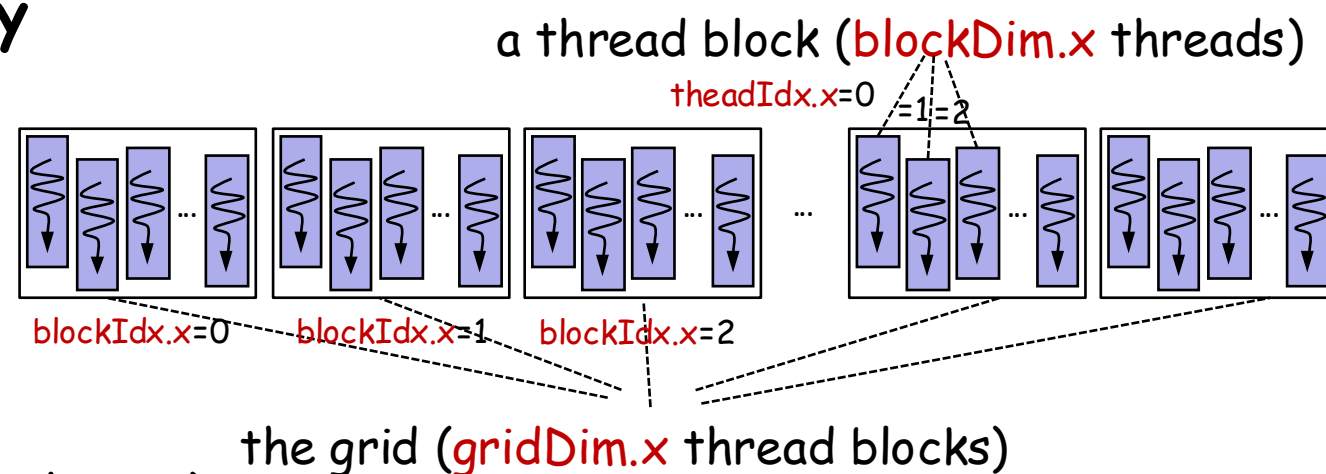
```
h0 () ;  
g0<<<...,>>>() ;  
h1 () ;  
g1<<<...,>>>() ;  
h2 () ;  
g2<<<...,>>>() ;  
cudaDeviceSynchronize() ;  
h3 () ;
```

- `g0` may overlap with `h1` and `h2`
- `g0` and `g1` do not overlap because of GPU serializes them by default
- `h3` does not overlap with anything because of `cudaDeviceSynchronize()`

About thread IDs

- for each thread to determine what to do, it needs a unique ID (the loop index)
- you get it from **gridDim**, **block{Dim,Idx}** and **threadIdx**
- when you launch a kernel by

```
f<<<nb,ns>>>(...);
```



- **blockDim.x** = bs (the thread block size)
- **gridDim.x** = nb (the number of blocks = the "grid" size) and
- **threadIdx.x** = the thread ID within the block ($\in [0, bs)$)
- **blockIdx.x** = the thread's block ID ($\in [0, nb)$)

nb and *bs* can be 2/3-Dimensional

- as suggested by *.x*, a block and the grid can be multidimensional (up to 3D, of *.x*, *.y*, *.z*) and the previous code assumes they are 1D
- extension to multidimensional block/grid is straightforward
- 1D:

```
int nb = 100;  
int bs = 256  
f<<<nb,bs>>>(...); /* 100*256 threads */
```

- 2D:

```
dim3 nb(10,10);  
dim3 bs(8,32);  
f<<<nb,bs>>>(...); /* 10*10*8*32 threads */
```

- 3D:

```
dim3 nb(10,5,2);  
dim3 bs(8,8,4);  
f<<<nb,bs>>>(...); /* 10*5*2*8*8*4 threads */
```

SpMV in CUDA

■ original serial code

```
for (k = 0; k < A.nnz; k++) {  
    i,j,Aij = A.elms[k];  
    y[i] += Aij * x[j];  
}
```

■ write a kernel that works on a single non-zero element

```
__global__ spmv_dev(A, x, y) {  
    k = blockDim.x * blockIdx.x + threadIdx.x; /* thread id */  
    if (k < A.nnz) {  
        i,j,Aij = A.elms[k];  
        y[i] += Aij * x[j]; } }
```

■ and launch it with $\geq$ nnz threads (we're not done yet)

```
spmv*(A, x, y) {  
    int bs = 256;  
    int nb = (A.nnz + bs - 1) / bs;  
    spmv_dev<<<nb,bs>>>(A, x, y); }
```

■ similarly simple for CSR version

We're not done yet

■ this code

```
__global__ spmv_dev(A, x, y) {  
    k = blockDim.x * blockIdx.x + threadIdx.x;  
    if (k < nnz) {  
        i,j,Aij = A.elms[k];  
        y[i] += Aij * x[j];  
    }  
}
```

■ does not work yet

1. the device **cannot access elements** of A, x and y on the host
2. there is a race condition when updating **y[i]**

Keywords for functions

■ `__global__`, `__device__`, `__host__`

	Callable from	Code runs on
<code>__global__</code>	host/device	device
<code>__device__</code>	device	device
<code>__host__</code>	host	host

- `__global__` functions cannot return a value (must be void)
- you can have both host and device in front of a definition, which generates two versions (device and host)

Macros

- convenient when writing a single file that works both on CPU and GPU
- `__NVCC__` : a macro defined when compiled by `nvcc`

```
#ifdef __NVCC__  
    /* GPU implementation */  
#else  
    /* CPU implementation */  
#endif
```

- `__CUDA_ARCH__`: a macro defined when compiled for device

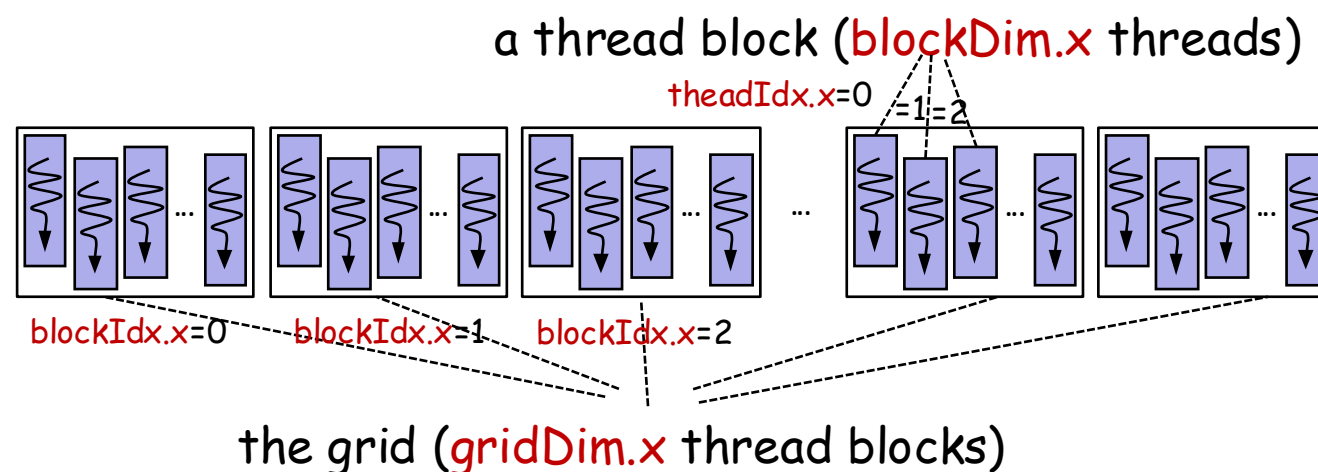
```
__device__ __host__ f(...) {  
#ifdef __CUDA_ARCH__  
    /* device code */  
#else  
    /* host code */  
#endif  
}
```


Contents

1. Overview
2. CUDA Basics
3. Kernels
- 4. Threads and thread blocks**
5. Communicating data between host and device
6. Data sharing among threads in the device
7. Choosing a block size

Threads and thread blocks (recap)

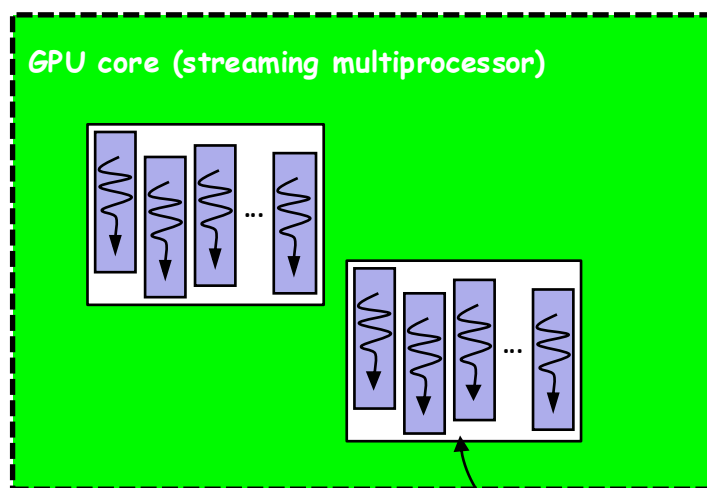
- a kernel specifies the action of a CUDA thread
- when you launch a kernel you specify
 - the number of thread blocks (**nb**) and
 - the thread block size = the number of threads in a single thread block (**bs**),
- to effectively create (**nb** $\times$ **bs**) threads



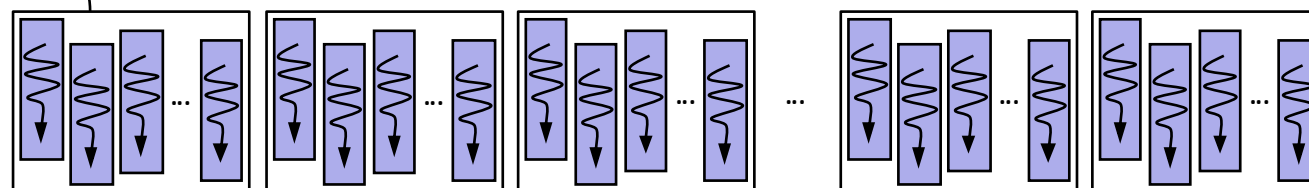
but why you need two separate numbers?

Why two numbers (bs and nb)?

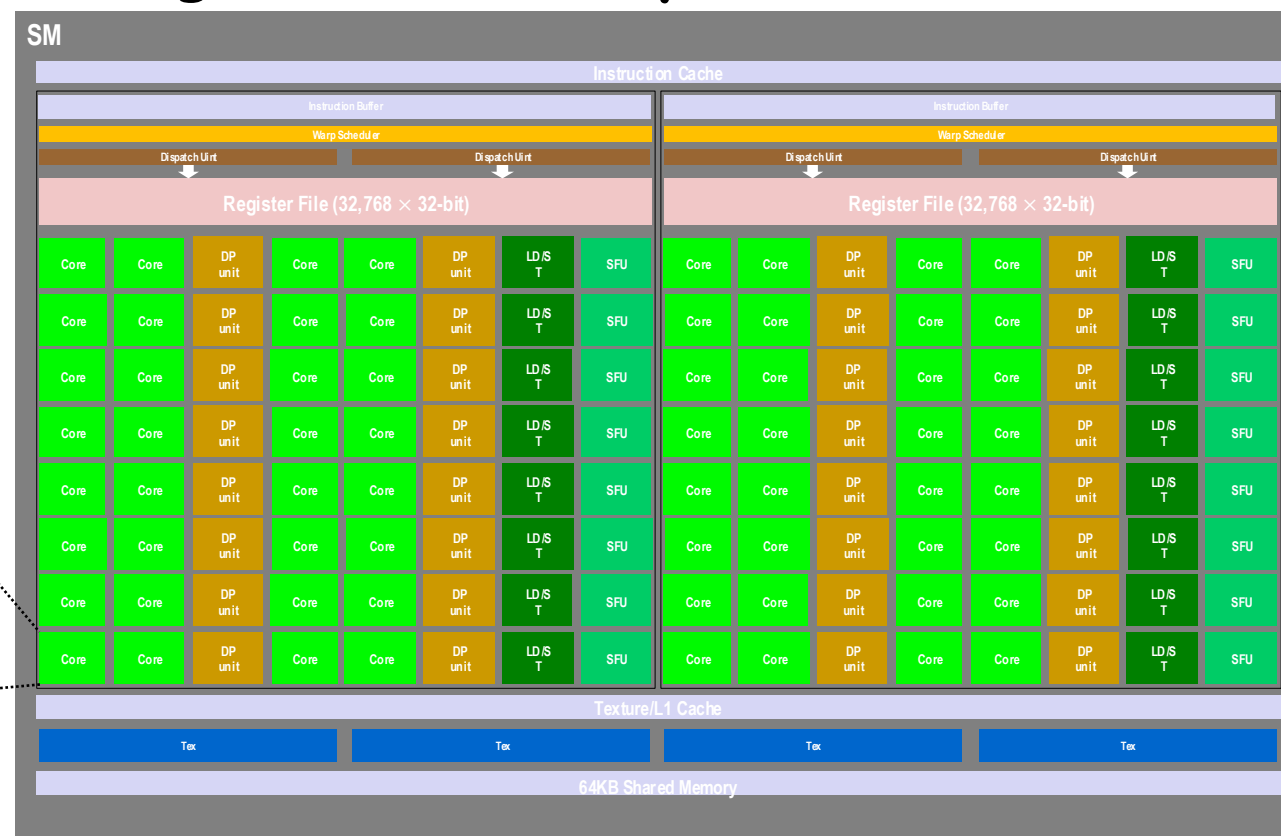
a single thread block is sent to a single SM and stays there until it finishes



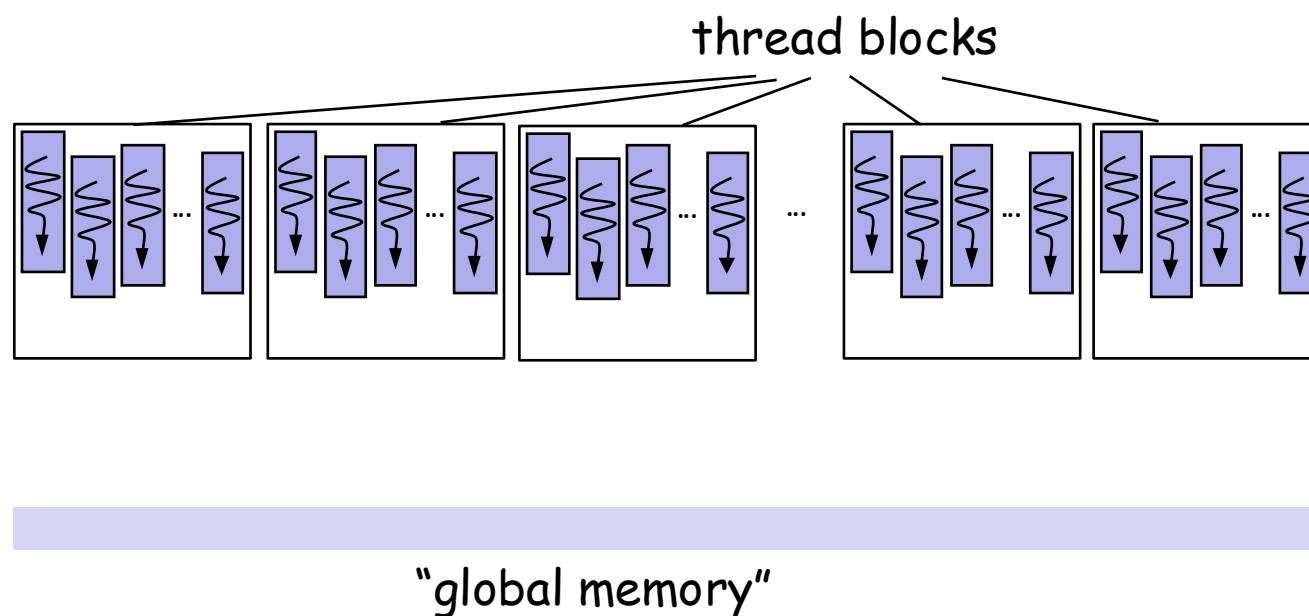
GPU sends a thread block to an SM



$f\langle\langle nb, bs \rangle\rangle(\dots)$

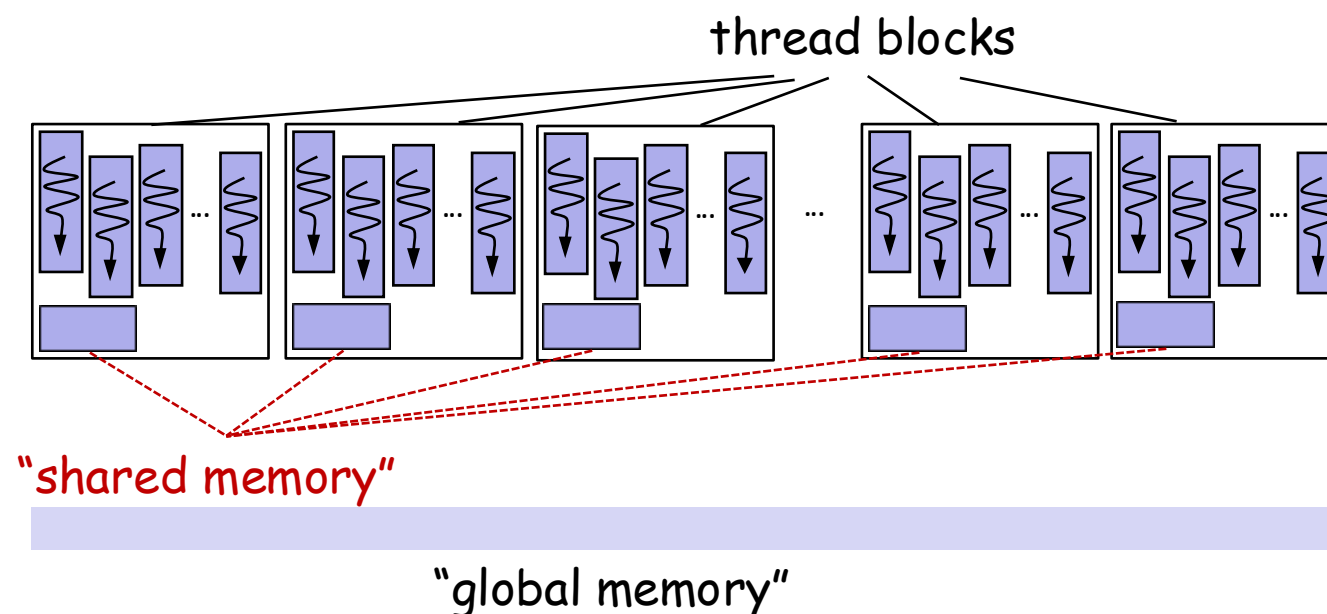


The way thread block boundaries are semantically visible



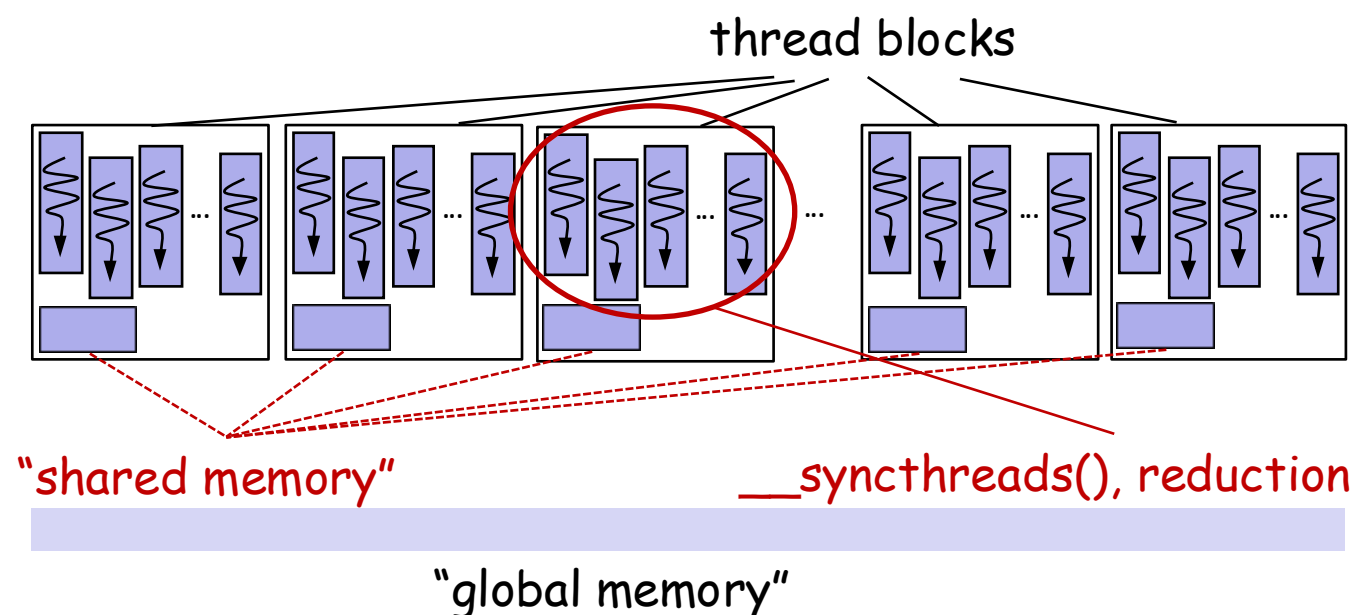
The way thread block boundaries are semantically visible

- CUDA API exposes “**shared memory**”, a small cache-like memory only shared within a single thread block



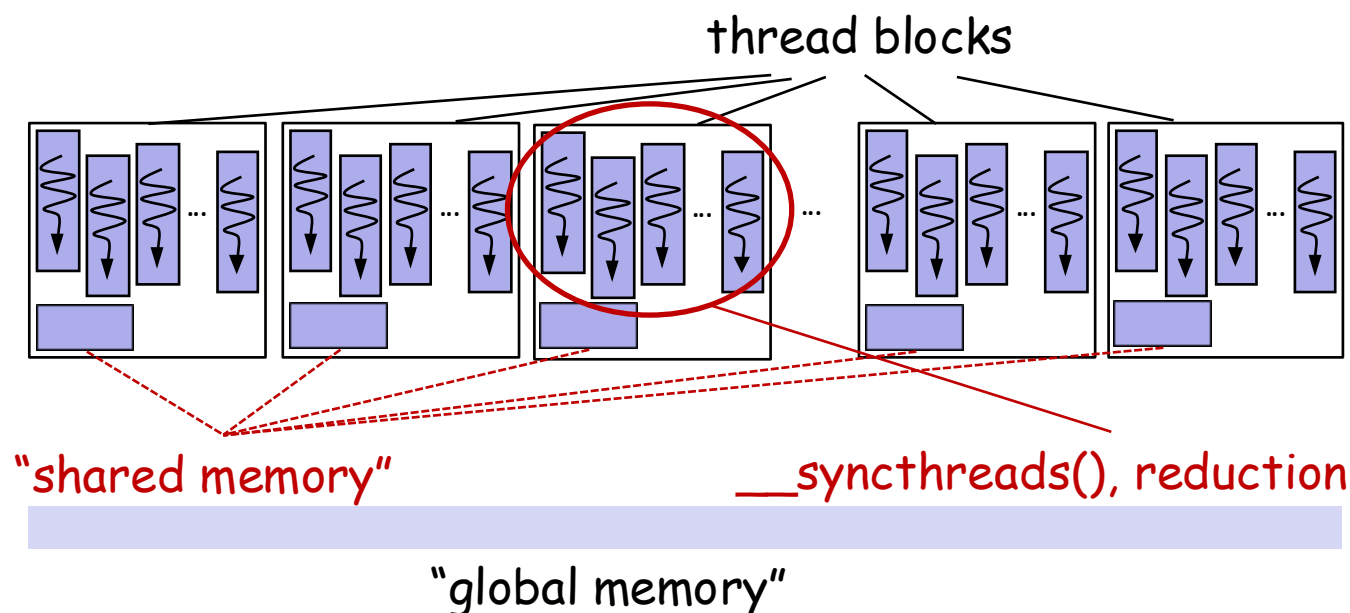
The way thread block boundaries are semantically visible

- CUDA API exposes “**shared memory**”, a small cache-like memory only shared within a single thread block
- CUDA API exposes some synchronization/coordination primitives (e.g., `syncthreads()` or reduction) only usable within a single thread block



The way thread block boundaries are semantically visible

- CUDA API exposes “**shared memory**”, a small cache-like memory only shared within a single thread block
- CUDA API exposes some synchronization/coordination primitives (e.g., `syncthreads()` or reduction) only usable within a single thread block
 - unless you rely on these primitives, choosing the thread block size is largely a performance (not a correctness) issue



Contents

1. Overview
2. CUDA Basics
3. Kernels
4. Threads and thread blocks
5. **Communicating data between host and device**
6. Data sharing among threads in the device
7. Choosing a block size

Moving data between host and device

- host and device memory are **separate**
- the device cannot access data on the host and vice versa (at least not directly by hardware until recently)
- i.e., the following does not work

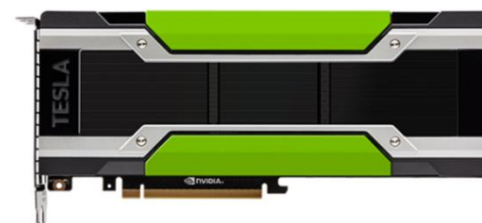
```
double a[n];  
f<<<nb,bs>>>(a);
```

```
__global__ f(double * a) {  
    ... a[i] ... /* this will segfault */  
}
```

host (CPU)



device (GPU)

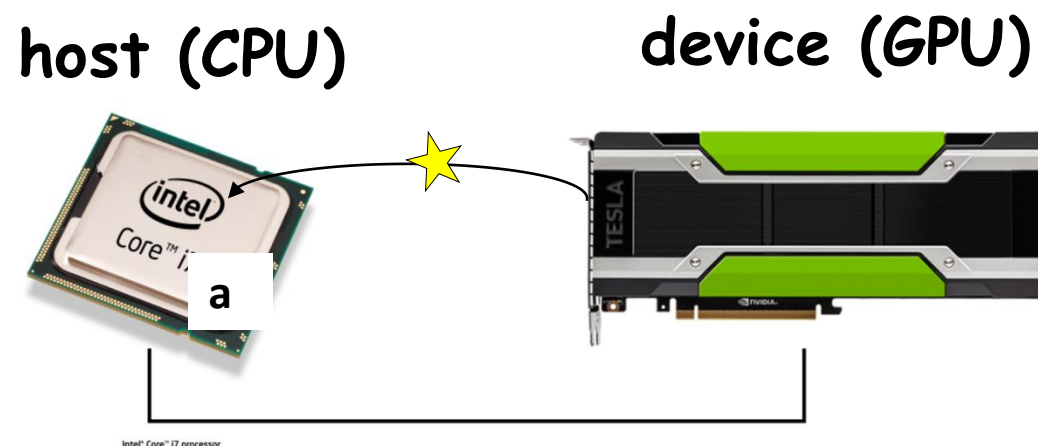


Moving data between host and device

- host and device memory are separate
- the device cannot access data on the host and vice versa (at least not directly by hardware until recently)
- i.e., the following does not work

```
double a[n];  
f<<<nb,bs>>>(a);
```

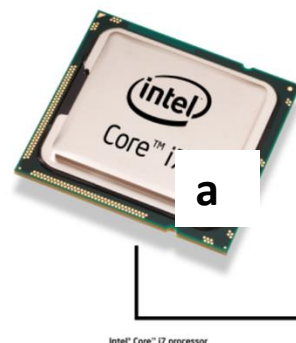
```
__global__ f(double * a) {  
    ... a[i] ... /* this will segfault */  
}
```



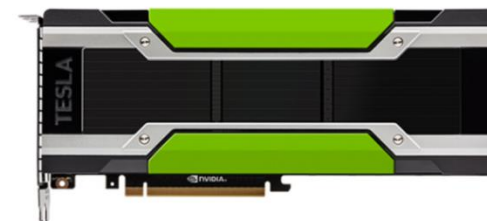
Two more things must master: cudaMalloc and cudaMemcpy

- you need to

host (CPU)



device (GPU)

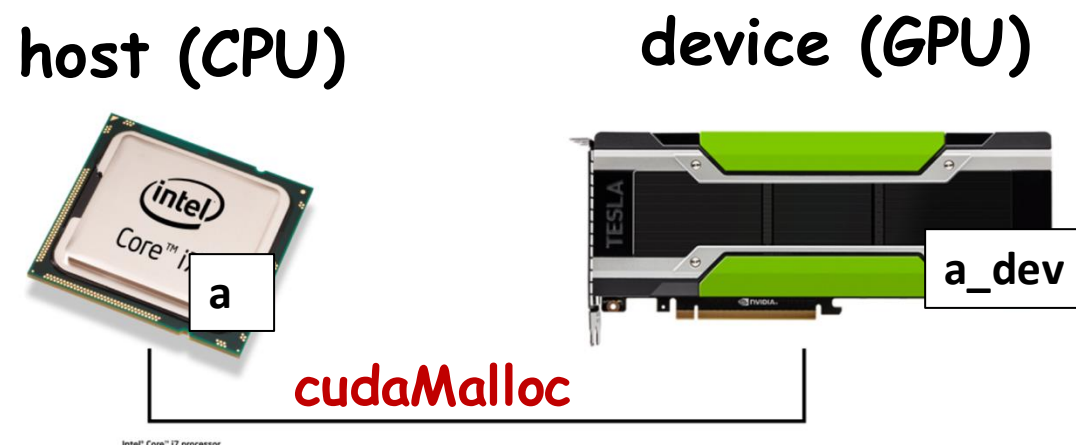


Intel® Core™ i7 processor

Two more things must master: cudaMalloc and cudaMemcpy

■ you need to

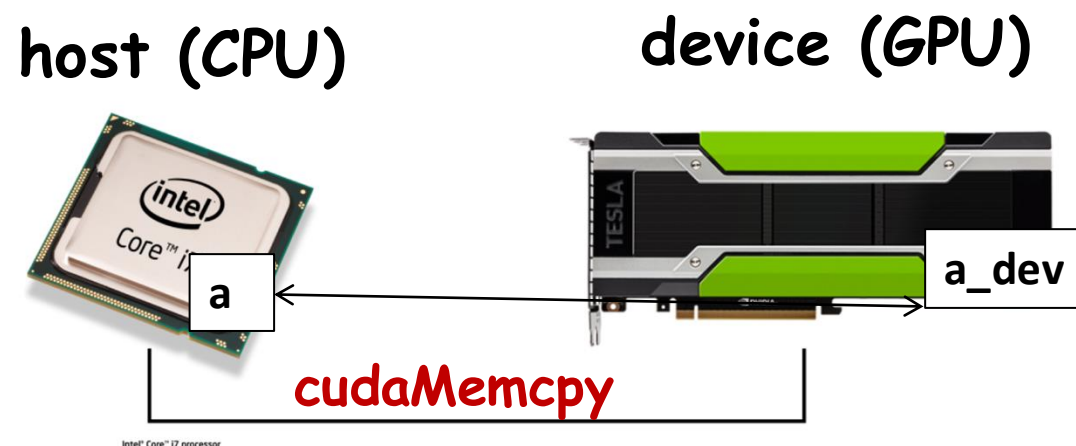
1. allocate data on device(by `cudaMalloc`) → *device memory*



Two more things must master: cudaMalloc and cudaMemcpy

■ you need to

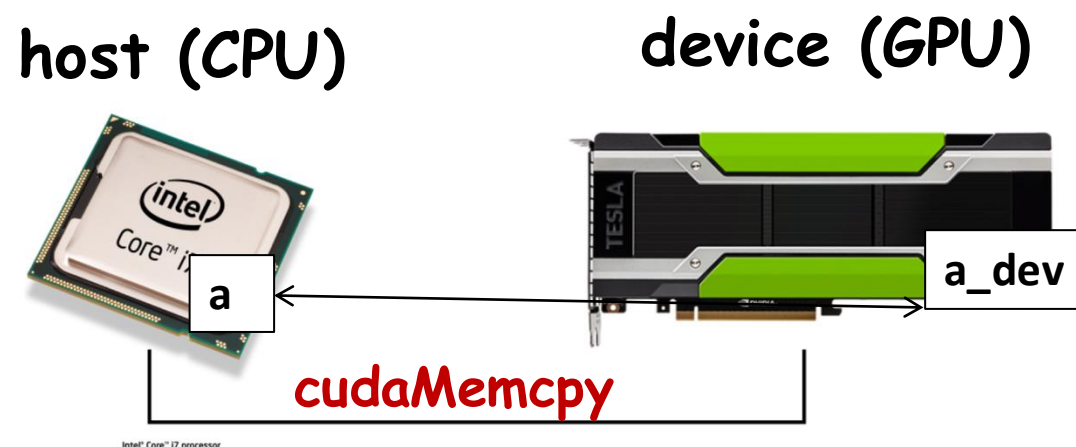
1. allocate data on device (by `cudaMalloc`) → *device memory*
2. move data between the host and the device (by `cudaMemcpy`)



Two more things must master: cudaMalloc and cudaMemcpy

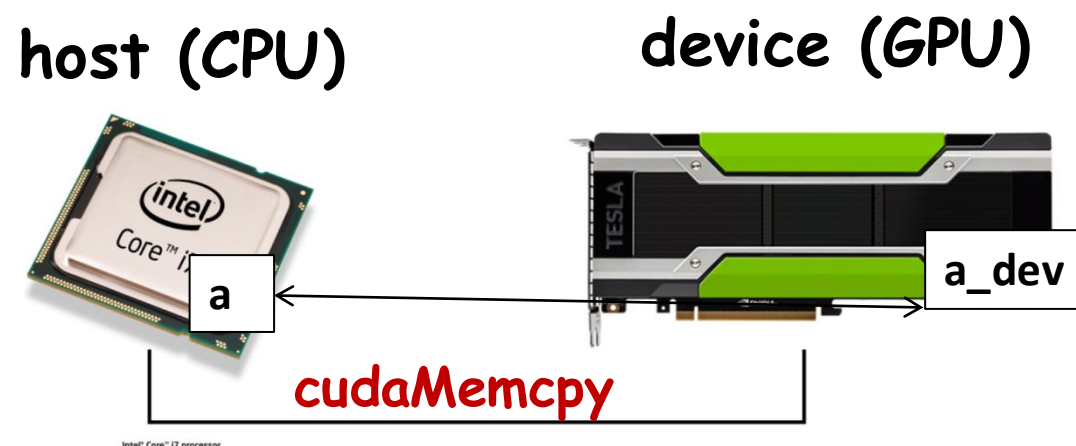
■ you need to

1. allocate data on device (by `cudaMalloc`) → *device memory*
2. move data between the host and the device (by `cudaMemcpy`)
3. give the kernel the pointer to the device memory



Two more things must master: cudaMalloc and cudaMemcpy

- you need to
 1. allocate data on device (by `cudaMalloc`) → *device memory*
 2. move data between the host and the device (by `cudaMemcpy`)
 3. give the kernel the pointer to the device memory
- note: call `cudaMalloc` and `cudaMemcpy` on the host, not on the device



Typical steps to send data to the device

1. allocate data of the same size both on host and device

```
double * a = ...; /* any valid address will do (malloc, &variable, etc.) */  
double * a_dev = 0;  
cudaMalloc((void **) &a_dev, sz);
```

2. the host works on the host data

```
for ( ... ) { a[i] = ... } /* whatever initialization you need */
```

3. copy the data to the device

```
cudaMemcpy(a_dev, a, sz, cudaMemcpyHostToDevice);
```

4. pass the device pointer to the kernel

```
f<<<nb,bs>>>(a_dev, ...)
```

5. often a good idea to have a struct encapsulating both pointers

```
typedef struct {  
    double * a; /* host pointer */  
    double * a_dev; /* device pointer */  
    ... } my_struct;
```


Typical steps to retrieve the result

1. allocate data of the same size both on host and device

```
double * r = ... ;  
double * r_dev = 0;  
cudaMalloc((void **)&r_dev, sz);
```

2. pass the device pointer to the kernel

```
f<<<nb,bs>>>(...,r_dev)
```

3. copy the data to the host

```
cudaMemcpy(r, r_dev, sz, cudaMemcpyDeviceToHost);
```

Unified Memory

- recent NVIDIA GPUs support **Unified Memory** that
- eliminate the need for explicit data movement between host
- and device memory and dual pointer management
- at the heart of it is **cudaMallocManaged**, which is like
- cudaMalloc but is directly accessible from host CPU too

Typical steps to send data to the device with Unified Memory

1. allocate data of the same size both on host and device

```
double * a = 0;  
cudaMallocManaged((void **)&a, sz);
```

2. the host works on the host data

```
for ( ... ) { a[i] = ... } /* whatever initialization you need */
```

3. pass the pointer to the kernel

```
f<<<nb,bs>>>(a, ...)
```

Typical steps to send data to the device with Unified Memory

1. allocate data with cudaMallocManaged

```
double * r = 0;  
cudaMallocManaged((void **)&r, sz);
```

2. pass the device pointer to the kernel

```
f<<<nb,bs>>>(..., r);
```

3. make sure threads finished their work

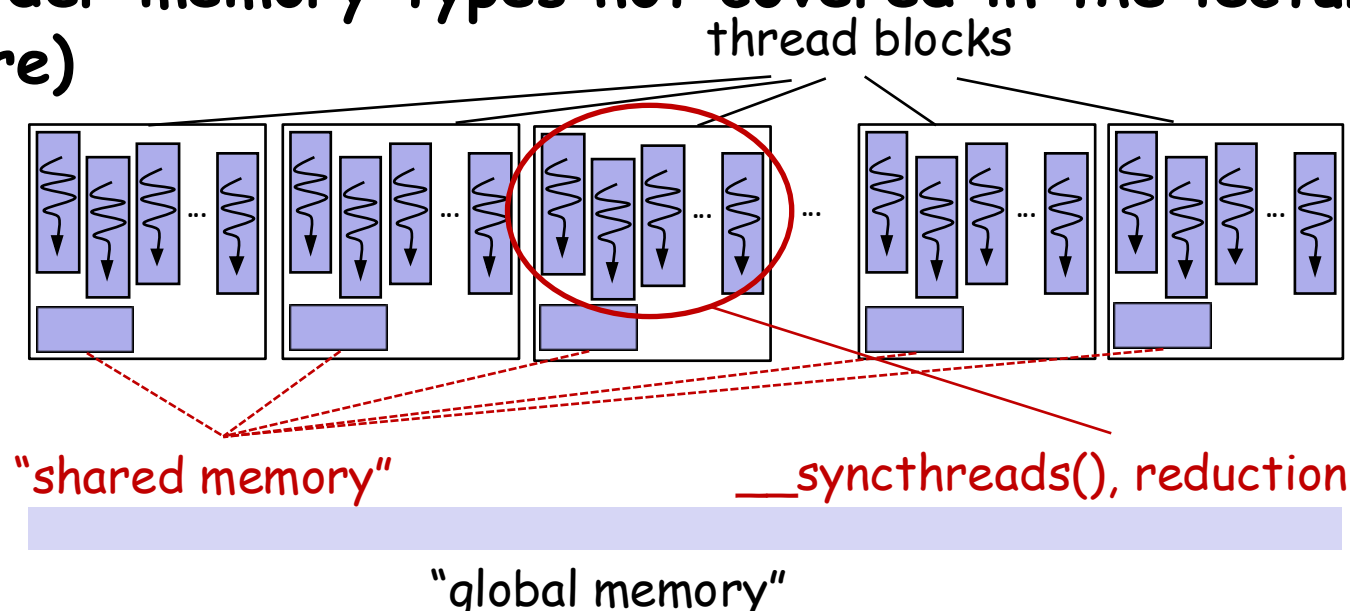
```
cudaDeviceSynchronize();
```

Contents

1. Overview
2. CUDA Basics
3. Kernels
4. Threads and thread blocks
5. Communicating data between host and device
- 6. Data sharing among threads in the device**
7. Choosing a block size

Data sharing among threads in the device

- basics : memory allocated via `cudaMalloc(Managed)?` are shared among all threads (*global memory*)
 - a write by a thread will be visible to all others (sooner or later)
- *shared memory* :
 - hardware terms: a small on-chip memory as fast as caches, not coherent across SMs
 - software view: memory shared only within a thread block
- other weirder memory types not covered in the lecture (constant and texture)



How to resolve race conditions on global/shared memory?

- CUDA threads run concurrently so they are susceptible to race conditions as in CPUs

```
__global__ spmv_dev(A, x, y) {  
    k = blockDim.x * blockIdx.x + threadIdx.x; /* thread id */  
    if (k < nnz) {  
        i,j,Aij = A.elems_dev[k];  
        y[i] += Aij * x[j];  
    }  
}
```

Resolving race condition on CUDA

- **atomic accumulations** (atomicAdd and other functions)
- forget about mutual exclusion (no #pragma omp critical equivalent)
- **barrier synchronization**, upon which you can built reductions
- **reduction**, but only within a single thread block

	OpenMP	CUDA
atomic accumulation	pragma atomic	atomicAdd
mutual exclusion	pragma critical or omp_lock_t	
barrier	pragma barrier	cooperative thread group
reduction	pragma reduction	only within a thread block, or DIY with barrier

Atomic accumulations

- consider the following (trivial) example

```
int * a;  
cudaMallocManaged(&a, sizeof(int) * nb * bs);  
for (i=0; i<nb*bs; i++) a[i] = 1;  
sum<<<nb,bs>>>(a);  
cudaDeviceSynchronize();  
printf("sum = %d\n", a[0]);
```

- the goal is to guarantee it always prints the sum of all elements in the array ($= nb \times bs$)
- a race-prone version

```
__global__ void f(int * a) {  
    int i = thread id;  
    if (i > 0) a[0] += a[i];  
}
```

Atomic accumulations

- atomic accumulations are supported by the hardware and CUDA API

```
#pragma omp atomic
*p += x
```

- in OpenMP
 - search the CUDA toolkit documentation for "atomicAdd"
- there are other primitives, such as compare-and-swap
- fix our example

```
__global__ void f(int * a) {
    int i = thread id;
    if (i > 0) atomicAdd(&a[0], a[i]);
}
```

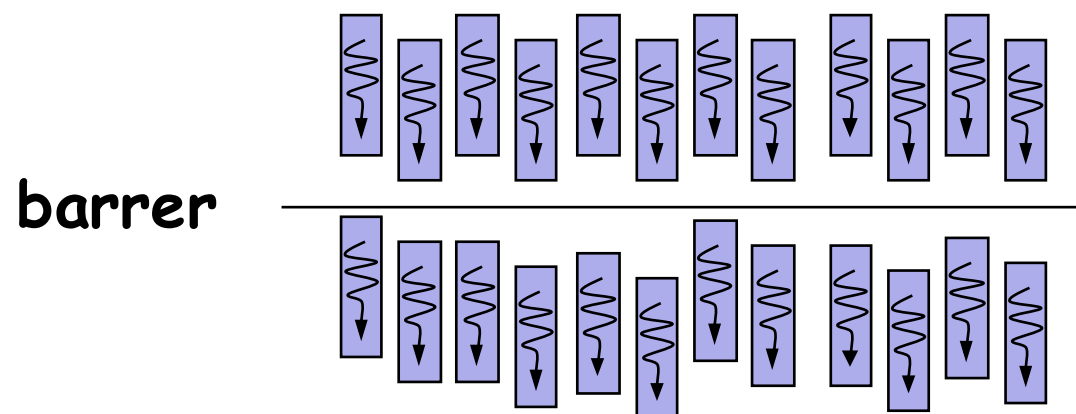
A working version of COO SpMV

```
__global__ spmv_dev(A, x, y) {  
    k = thread id;  
    if (k < nnz) {  
        i,j,Aij = A.elems_dev[k];  
        atomicAdd(&y[i], Aij * x[j]);  
    }  
}
```

- make sure `A.elems_dev`, `x` and `y` point to device memory (not shown)
- note: CSR is simpler to work with if you don't parallelize within a row

Barrier synchronization

- barrier is a mechanism to ensure “all threads reached a point”
- useful to ensure changes made by a thread is visible all others
- CUDA used to support barriers only within a single thread block (`__syncthreads()`)
- it now supports barriers for all threads (C. Cooperative Groups)



Cooperative groups (1)

- (important) when using the following features, launch a kernel by

```
void * args[] = { a0, a1, ... };  
cudaLaunchCooperativeKernel((void *)f, nb, bs, args);
```

- instead of the ordinary

```
f<<<nb,bs>>>(a0, a1, ...);
```

- common setup

```
#include <cooperative_groups.h>  
namespace cg = cooperative_groups; /* save typing */
```

Cooperative groups (2)

■ data representing a group

```
cg::grid_group g = cg::this_grid(); /* all threads */
```

```
cg::thread_block g = cg::this_thread_block(); /* thread block */
```

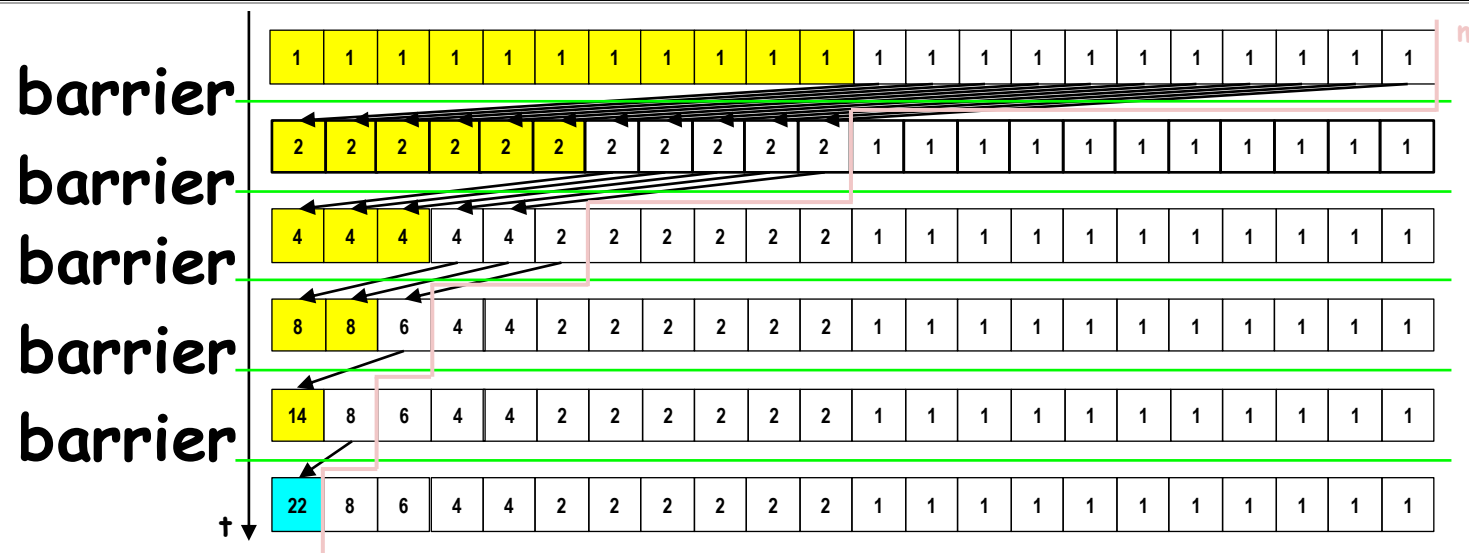
■ barrier synchronization

```
g.sync(); /* barrier all theads in g participate */
```

■ group actually provides a cleaner way to know thread ID and number of threads

```
unsigned long long idx = g.thread_rank(); /* my ID in g */  
unsigned long long nth = g.size(); /* num threads in g */
```

Building reduction on barrier



```
__global__
void sum(double * c, long n) {
    /* return c[0] + .. + c[n-1] */
    cg::grid_group g = cg::this_grid();
    ull i = g.thread_rank();
    ull h; /* ull: unsigned long long */
    for (long m = n; m > 1; m = h) {
        h = (m + 1) / 2;
        if (i + h < m) c[i] += c[i + h];
        g.sync();
    }
}
```

- invariant: "sum($c[0:m]$) is the sum", it repeats halving m
- note: it may not be most efficient; reducing values within a single block first may be better

CUDA shared memory

- CUDA programs can allocate a “shared memory” to each thread

```
f<<<nb,bs,S>>>(. . .);
```

- from CUDA program's perspective, it is a memory **only shared within a thread block** and only active during the thread block's lifetime
 - the term **shared memory** is a misnomer, IMO; ordinary memory you allocate via `cudaMalloc` is shared by all threads
 - local memory or something will be a more appropriate name
- physically, it is a cache-like memory faster than global memory
- each SM has a fixed amount of shared memory (A100 : 164KB)
- $S \leq$ shared memory per SM

Accessing CUDA shared memory

- specify the shared memory size on a kernel call and a kernel accesses it by declaring variables or arrays with `__shared__`

```
__shared__ int a[n];  
__shared__ char b[m];
```

- if the data size (n or m above) is not a compile-time constant, obtain the starting address of the shared memory by

```
extern __shared__ char whatever[];
```

- it's your responsibility to use appropriate part of it. e.g.,

```
int * a = (int*)whatever;  
char * b = (char *)&a[n];
```

- shared memory is a way to efficiently communicate among threads within a block
- GPU has nowadays processor-managed caches, so how crucial it is to performance is somewhat changing over time

Contents

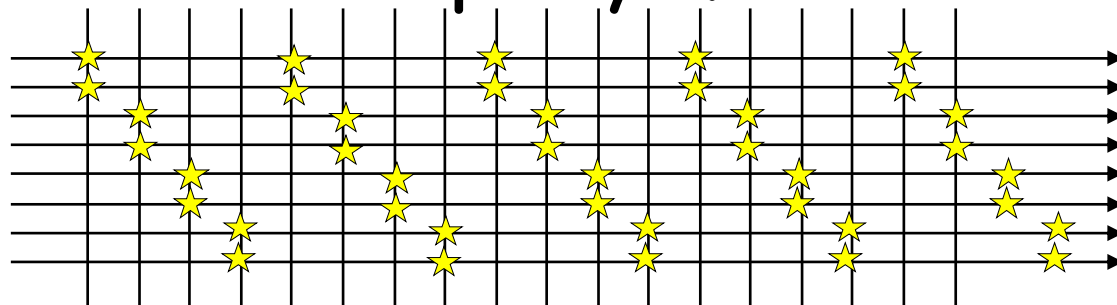
1. Overview
2. CUDA Basics
3. Kernels
4. Threads and thread blocks
5. Communicating data between host and device
6. Data sharing among threads in the device
7. **Choosing a block size**

Choosing a good block size for performance

- the question is, when you create a number of, say 10000, threads, how you divide them into thread blocks?

```
f<<<1, 10000>>>(x, y, z, ...);  
f<<<10, 1000>>>(x, y, z, ...);  
f<<<100, 100>>>(x, y, z, ...);  
f<<<1000, 10>>>(x, y, z, ...);  
f<<<10000, 1>>>(x, y, z, ...);
```

- and countless other ways ...
- the goal is to run a sufficient number of threads **simultaneously** so they utilize the hardware capacity of an SM



- to this end, let's understand what a GPU actually does, given a thread block size and the number of blocks

What limits the number of threads that can run simultaneously?

- **registers** used by simultaneously running threads must fit registers of an SM
- **shared memory** used by simultaneously running thread blocks must fit shared memory of an SM
- they, along with other trivial limits, together limit/determine the number of threads that can run simultaneously on a single SM

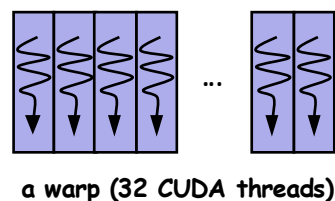
Parallelism within an SM

- consists of three levels
- **thread $\subset$ warp $\subset$ thread block $\subset$ SM**



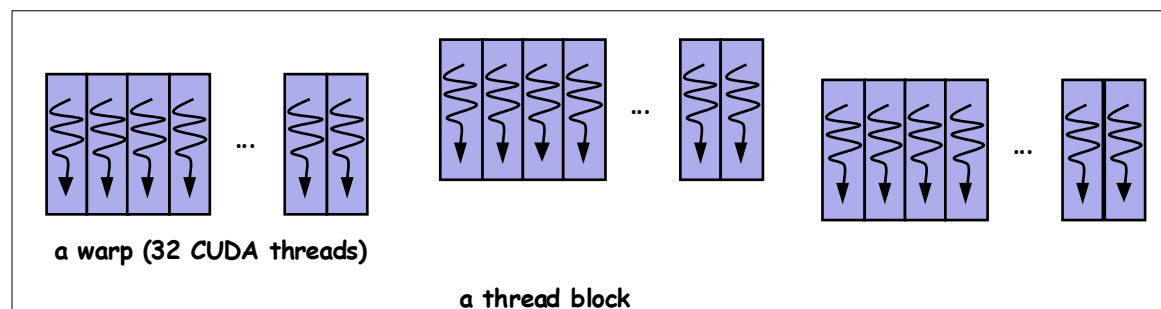
Parallelism within an SM

- consists of three levels
- **thread** $\subset$ **warp** $\subset$ **thread block** $\subset$ **SM**
- a group of **32** CUDA threads makes a **warp**



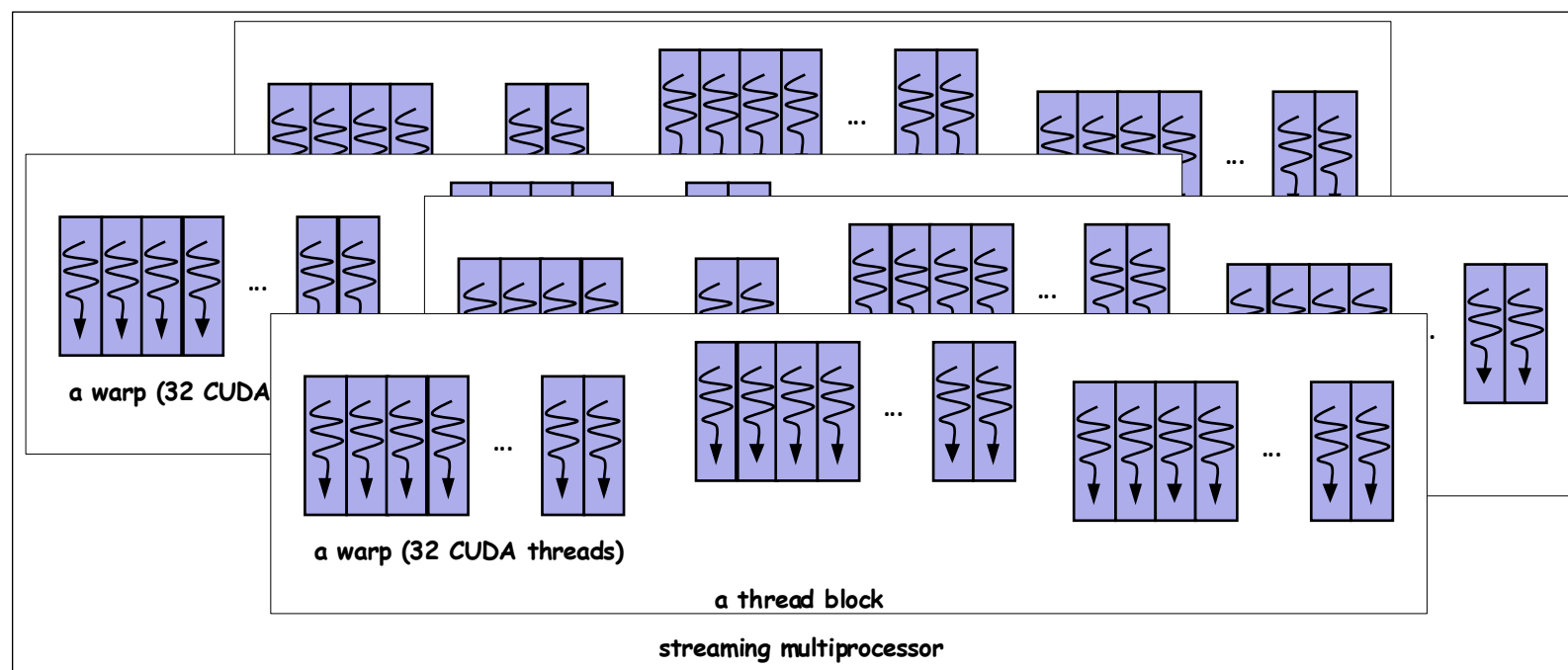
Parallelism within an SM

- consists of three levels
- **thread** $\subset$ **warp** $\subset$ **thread block** $\subset$ **SM**
- a group of **32** CUDA threads makes a **warp**
- a group of **$\lceil bs/32 \rceil$** warps makes a **thread block**



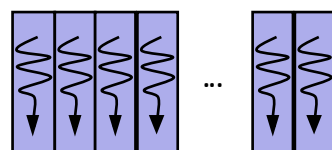
Parallelism within an SM

- consists of three levels
- **thread** $\subset$ **warp** $\subset$ **thread block** $\subset$ **SM**
- a group of **32** CUDA threads makes a **warp**
- a group of **$\lceil bs/32 \rceil$** warps makes a **thread block**
- and there are multiple thread blocks executing simultaneously on a single SM



Warps

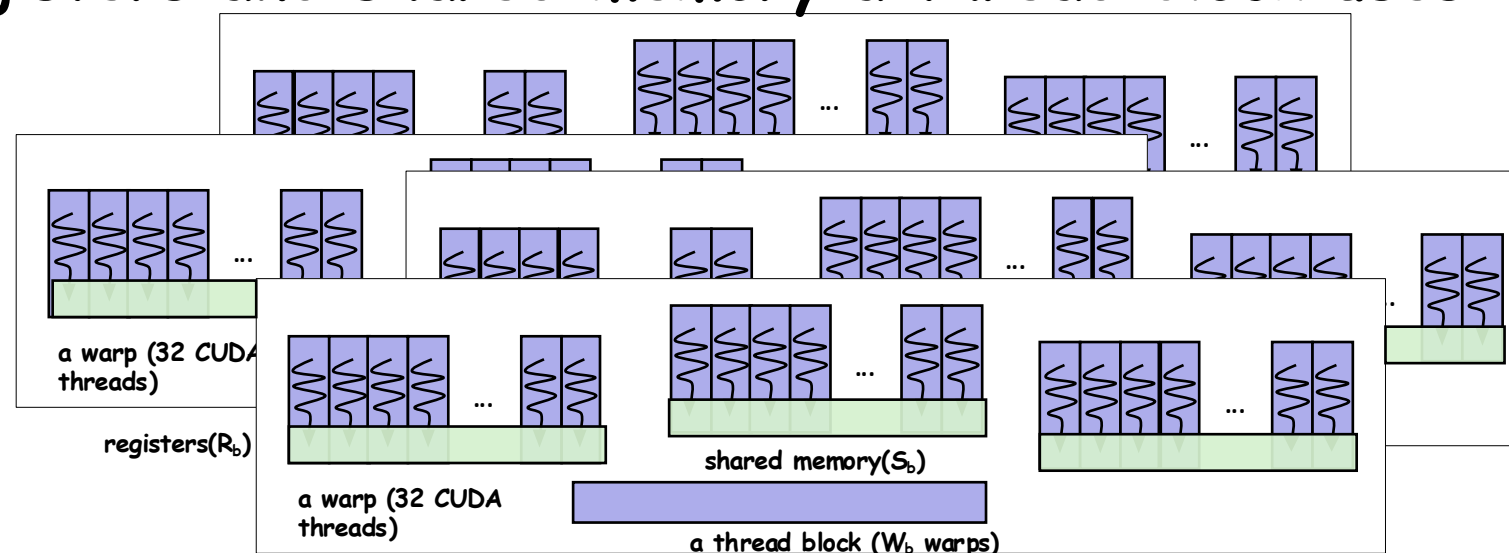
- a warp is the unit of **instruction execution**
- 32 threads in a single warp share an instruction pointer (a warp $\approx$ a CPU thread executing 32-way SIMD instructions)
- at every cycle, an SM selects a few (actually, ≤ 2) warps and execute them
- $\Rightarrow$
 - there is rarely a point in making $bs < 32$ or not a multiple of 32 (remainder threads consume resources but perform no useful work)
 - you want to make 32 threads branch in the same way (avoid warp divergence)



a warp (32 CUDA threads)

Thread blocks

- a thread block is **the unit of dispatching to an SM**
- conceptually, a kernel launch $f\langle\langle nb, \dots \rangle\rangle(x, y, \dots)$ puts nb blocks in a queue, which GPU dispatches to SMs, one block at a time
- once a block starts running, they stay on the SM **until it finishes** and occupies **registers** and **shared memory** throughout
- $\Rightarrow$ the number of blocks simultaneously running on an SM is limited by registers and shared memory a thread block uses



Registers and shared memory

■ registers

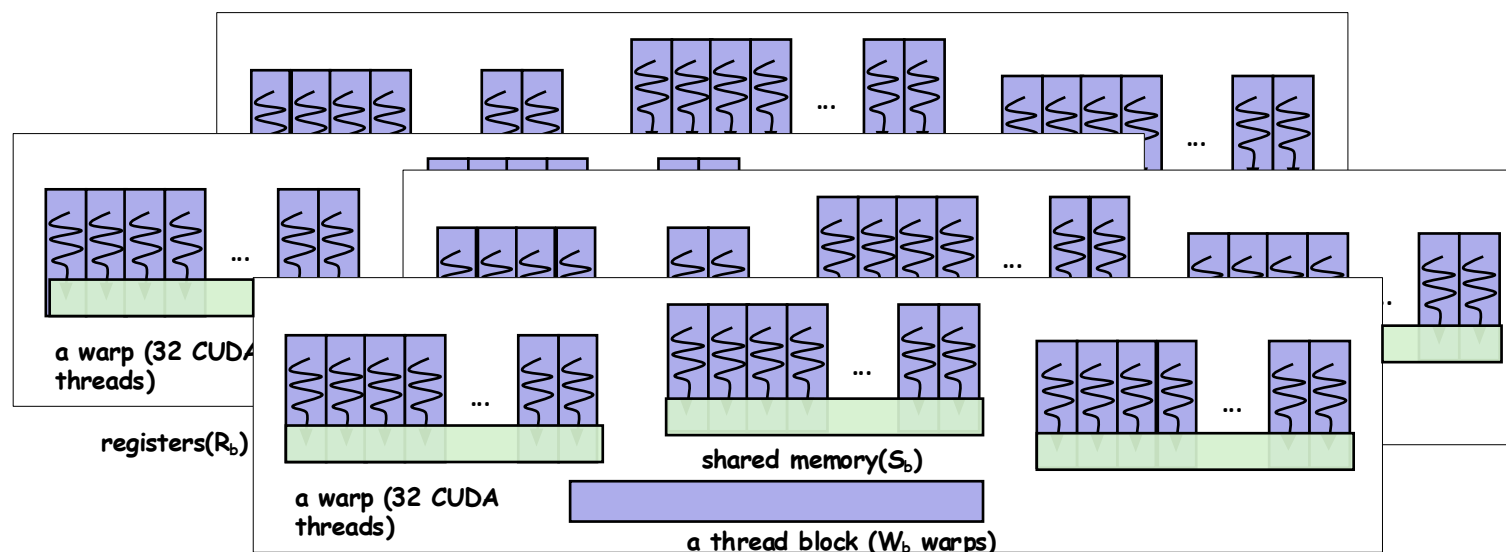
- hold **local and temporary** variables of threads
- the size is determined by your program and the compiler

■ shared memory

- can be allocated when launching a kernel by

```
f<<<nb, bs, Sb>>>(x, y, z, ...)
```

- and is shared within a thread block



Hardware limits

- all numbers are per SM
- **A100 (compute capability 8.0)**

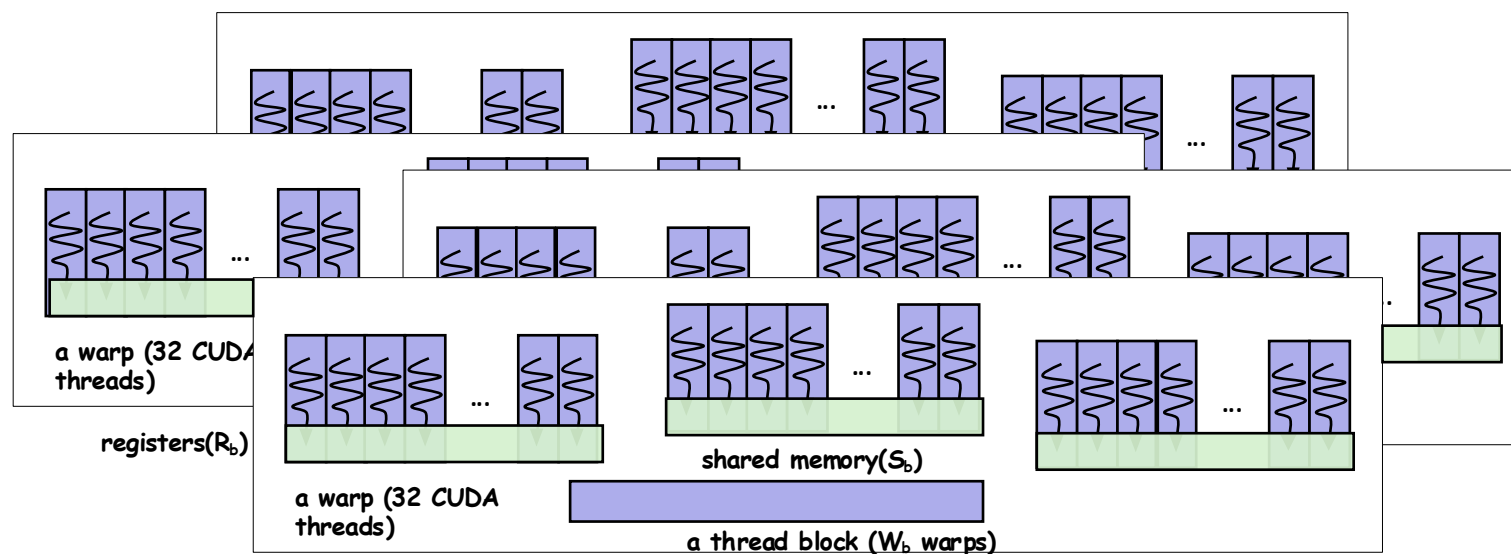
registers	65336 × 32 bits
shared memory	164KB
warps that can simultaneously run	64
thread blocks that can simultaneously run	32

- **V100 (compute capability 7.0)**

registers	65336 × 32 bits
shared memory	96KB
warps that can simultaneously run	64
thread blocks that can simultaneously run	32

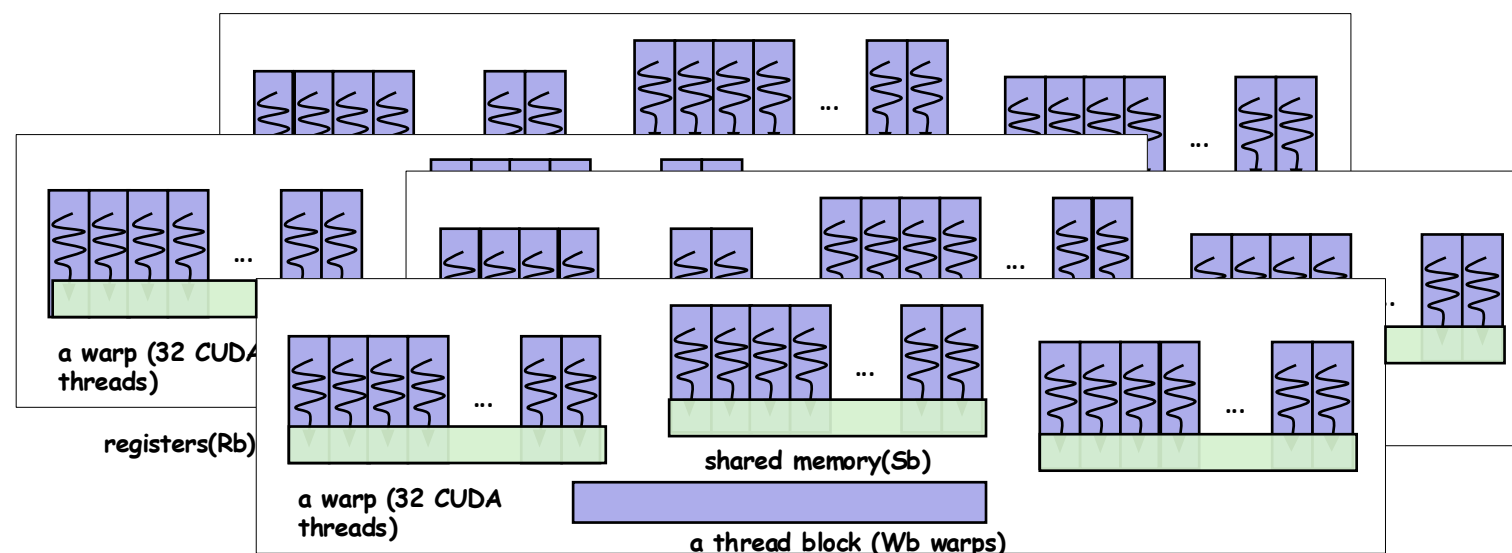
Putting them together

- blocks that will simultaneously run on an SM
- given (from the programmer or the compiler)
 - T_b : the number of threads per block,
 - S_b : shared memory size per block, and
 - R_1 : registers per thread,
- calculate various resources per block
 - warps per block : $W_b = \lceil T_b / 32 \rceil$
 - registers per block : $R_b = 32R_1 \times W_b$



Putting them together

- the number of blocks that simultaneously run on an SM (nb)
- $nb = \min(\lfloor 65536/R_b \rfloor, \lfloor 164K/S_b \rfloor, \lfloor 64/W_b \rfloor, 32) = \min(\lfloor 2048/(R_1 \cdot W_b) \rfloor, \lfloor 164K/S_b \rfloor, \lfloor 64/W_b \rfloor, 32)$
- the number of warps simultaneously run on an SM (nw)
- $nw = W_b \cdot nb = W_b \cdot \min(\lfloor 2048/(R_1 \cdot W_b) \rfloor, \lfloor 164K/S_b \rfloor, \lfloor 64/W_b \rfloor, 32)$



Takeaways (often good thread block sizes)

$$W_b \cdot \min(\lfloor 2048/(R \cdot W_b) \rfloor, \lfloor 164K/S_b \rfloor, \lfloor 64/W_b \rfloor, 32)$$

if we ignore factors that come from R_1 and S_b , a guideline is to run the maximum 64 warps simultaneously and it can be accomplished by

- putting at least two warps in a block (so $64/W_b \leq 32$) and
- choosing the number of warps per block that divides 64
- that is, $W_b = 2, 4, 8, 16, 32$ (or $T_b = 64, 128, 256, 512, 1024$)

Remarks

- **64 warps is merely an upper bound that**
 - may not be necessary to get the maximum performance (e.g., floating point performance, whose limit is 2-warp (= 64) FMAs per cycle) and
 - may not be achievable due to other constraints (registers and shared memory)
- **the above takeaway is a rule of thumb to eliminate bad thread block sizes**

Occupancy calculator

- NVIDIA used to provide a simple Excel to give you how many warps can run simultaneously given block size (T_b), shared memory per block (S_b), and registers per thread (R_1)
- a small web page doing the same at <https://xmartlabs.github.io/cuda-calculator/>